Suivi de l’ISS sur ESP32

Objet connecté • MicroPython • Écran OLED 128 × 64 • API Open Notify

Khalid Asseffar & Meryem
Institut Grasset — 420-PC6-AG, groupe 00098
Programmation d’objets connectés — Fernando Ribeiro Trovão
Remise et présentation : 8 octobre 2026

VERSION FINALE VALIDÉE Le prototype a été testé sur l’ESP32 réel : OLED détecté à 0x3C, connexion Wi-Fi réussie, données de l’ISS reçues et affichage de la carte en fonctionnement.

01 / Objectif et fonctionnement général

Le but du projet est de réaliser un objet connecté capable d’afficher en temps réel la position de la Station spatiale internationale (ISS). L’ESP32 se connecte au réseau Wi-Fi, interroge l’API publique Open Notify, extrait la latitude et la longitude de la réponse JSON, transforme ces valeurs en coordonnées d’écran puis affiche un marqueur sur une carte du monde simplifiée.

Étape	Bloc	Rôle
1	Wi-Fi	Établir ou conserver la connexion réseau.
2	API	Envoyer une requête GET vers ISS Location Now.
3	JSON	Valider la réponse et convertir latitude / longitude en float.
4	Projection	Transformer les degrés en pixels (x, y).
5	OLED	Redessiner la carte et placer le X de l’ISS.

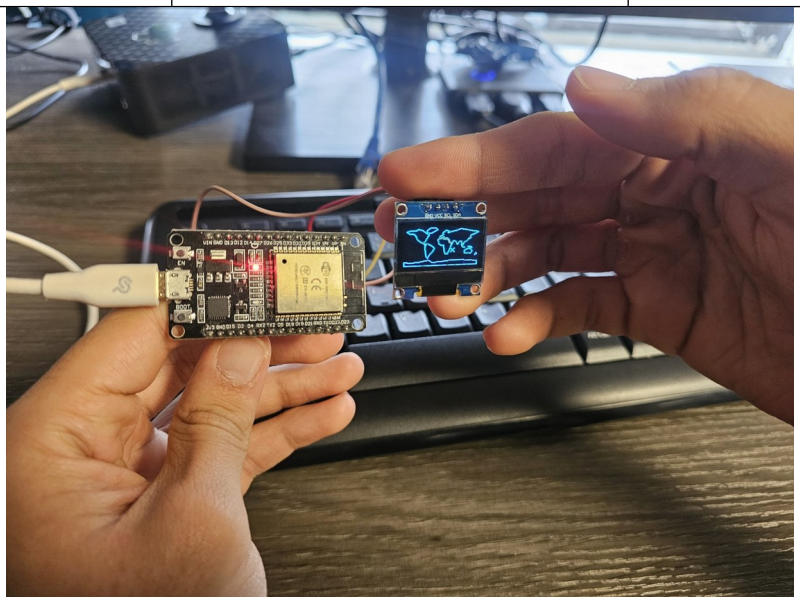


Figure 1 — Prototype réel : ESP32 et écran OLED affichant le planisphère.

02 / Matériel, câblage et organisation du code

Le montage utilise un ESP32 avec MicroPython et un écran OLED monochrome SSD1306 de 128 × 64 pixels. La communication entre l'ESP32 et l'écran se fait par le bus I2C. Le scan réalisé pendant les essais a détecté l'écran à l'adresse 0x3C.

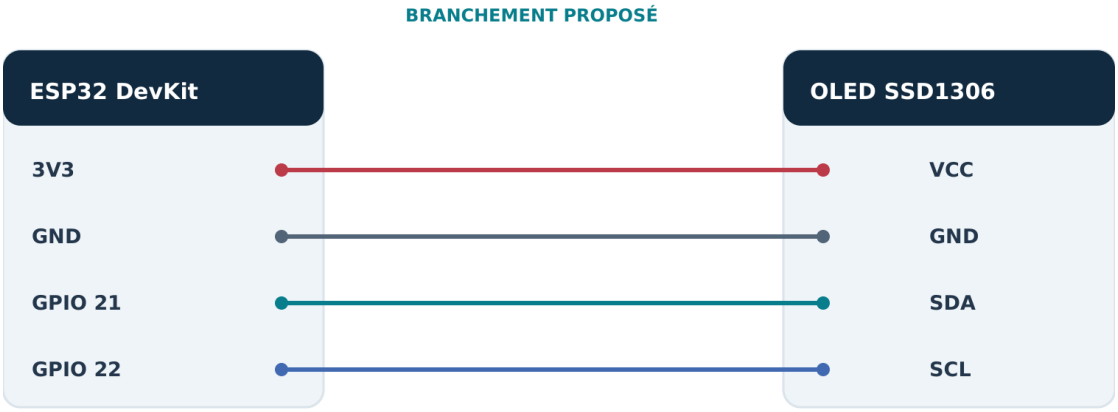


Figure 2 — Schéma de branchement ESP32 - OLED SSD1306.

ESP32	OLED	Fonction
3V3	VCC	Alimentation 3,3 V
GND	GND	Masse commune
GPIO 21	SDA	Ligne de données I2C
GPIO 22	SCL	Horloge I2C

Organisation logicielle

Le code est séparé en modules afin d'isoler les responsabilités et de faciliter les tests. Cette structure évite de mélanger la logique réseau, les calculs et l'affichage.

Fichier	Responsabilité
main.py	Boucle principale, coordination, cadence et gestion globale des erreurs.
iss_api.py	Connexion Wi-Fi, requête HTTP, validation JSON et extraction des coordonnées.
display.py	Initialisation OLED, conversion latitude/longitude et dessin du marqueur.
world_map.py	Points simplifiés servant à tracer les contours des continents.
settings.py	URL, broches, dimensions, adresse I2C et délais.
ssd1306.py / urequests.py	Pilote OLED et bibliothèque de requêtes HTTP.

PARAMÈTRES FINAUX OLED 0x3C • I2C 100 kHz • 128 × 64 pixels • cadence ≈ 3 s • délai Wi-Fi 30 s • délai HTTP 10 s • config.py local non remis.

03 / Connexion Wi-Fi, API et gestion des erreurs

Au démarrage, main.py importe les identifiants depuis config.py, initialise l'OLED puis appelle connect_wifi(). Si une connexion est déjà active, elle est conservée. Sinon, l'ESP32 tente de se connecter pendant une période maximale de 30 secondes. En cas d'échec, la boucle principale affiche un message et réessaie au cycle suivant.

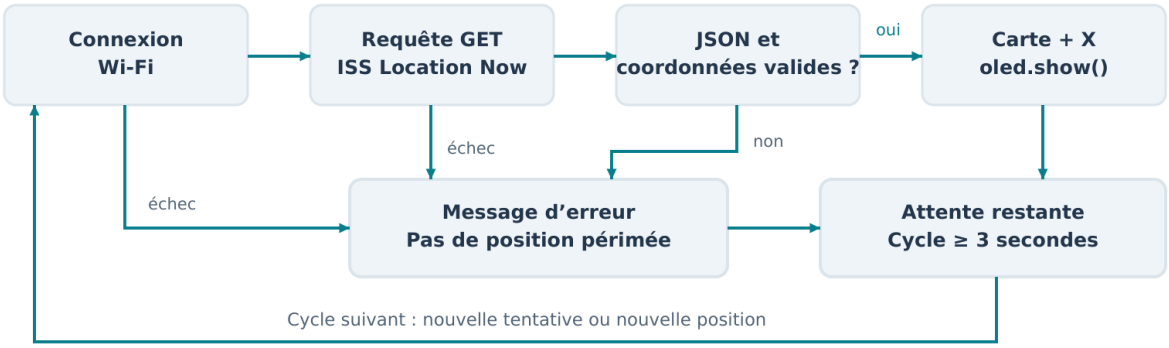


Figure 3 — Cycle principal : connexion, requête, validation, affichage et nouvelle tentative.

Appel HTTP et traitement JSON

REQUÊTE GET http://api.open-notify.org/iss-now.json

get_iss_position() attend un statut HTTP 200, un champ message égal à « success » et un objet iss_position. Les valeurs de latitude et de longitude sont reçues sous forme de texte ; elles sont converties avec float() puis vérifiées avant utilisation.

{"message":"success", "iss_position":{"latitude":"45.5017", "longitude":"-73.5673"}}

Situation	Réponse du programme
Wi-Fi indisponible	Aucun appel API ; message d'état puis nouvelle tentative.
HTTP / JSON incorrect	Réponse rejetée ; aucune fausse position n'est affichée.
Coordonnées invalides	Valeurs non numériques ou hors limites rejetées.
ETIMEDOUT	Exception capturée ; le programme continue au cycle suivant.

ROBUSTESSE OBSERVÉE Lors des essais, une erreur [Errno 116] ETIMEDOUT est apparue. Les requêtes suivantes ont de nouveau fourni des coordonnées valides : le système récupère donc automatiquement après un délai réseau.

04 / Projection géographique et affichage sur l'OLED

L'écran n'utilise pas des degrés mais des pixels. La projection équirectangulaire demandée répartit les longitudes de -180° à +180° sur les 128 colonnes et les latitudes de +90° à -90° sur les 64 lignes.

Coordonnée horizontale

$$x = (longitude + 180) \times 128 / 360$$

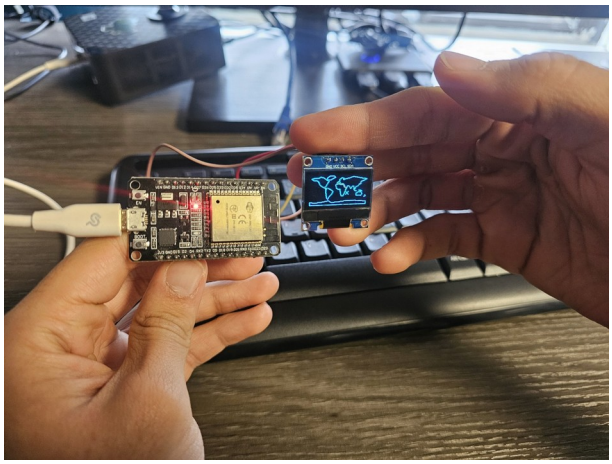
Coordonnée verticale

$$y = (90 - latitude) \times 64 / 180$$

coordinates_to_pixels() convertit ensuite le résultat en entier et borne les valeurs pour rester dans l'écran : x de 0 à 127 et y de 0 à 63. Le haut de l'écran correspond au nord ; y augmente vers le bas.

Latitude	Longitude	Pixel obtenu
0°	0°	(64, 32) — centre
-10,5895°	-160,3055°	(7, 35) — mesure réelle observée
-14,6219°	49,8157°	(81, 37) — mesure réelle observée
-18,2043°	52,6820°	(82, 38) — déplacement observé

Dessin de la carte et du marqueur



Méthode utilisée

• world_map.py contient des contours simplifiés sous forme de couples (longitude, latitude).

• draw_map() convertit chaque point en pixel puis relie les points avec oled.line().

• À chaque mise à jour, l'écran est effacé, la carte est redessinée, puis le marqueur est ajouté en dernier.

• Le X a été légèrement renforcé pour rester visible lorsqu'il passe sur un contour.

Figure 4 — Carte du monde affichée sur le véritable OLED pendant les essais.

REMARQUE Plusieurs positions successives peuvent donner le même pixel. Avec seulement 128 × 64 pixels, une petite variation géographique n'est pas toujours suffisante pour déplacer immédiatement le marqueur.

05 / Validation expérimentale, problèmes et conclusion

La validation finale a été réalisée sur le matériel réel au collège. Les essais confirment le fonctionnement de toute la chaîne : ESP32 → Wi-Fi → API → JSON → conversion → OLED.

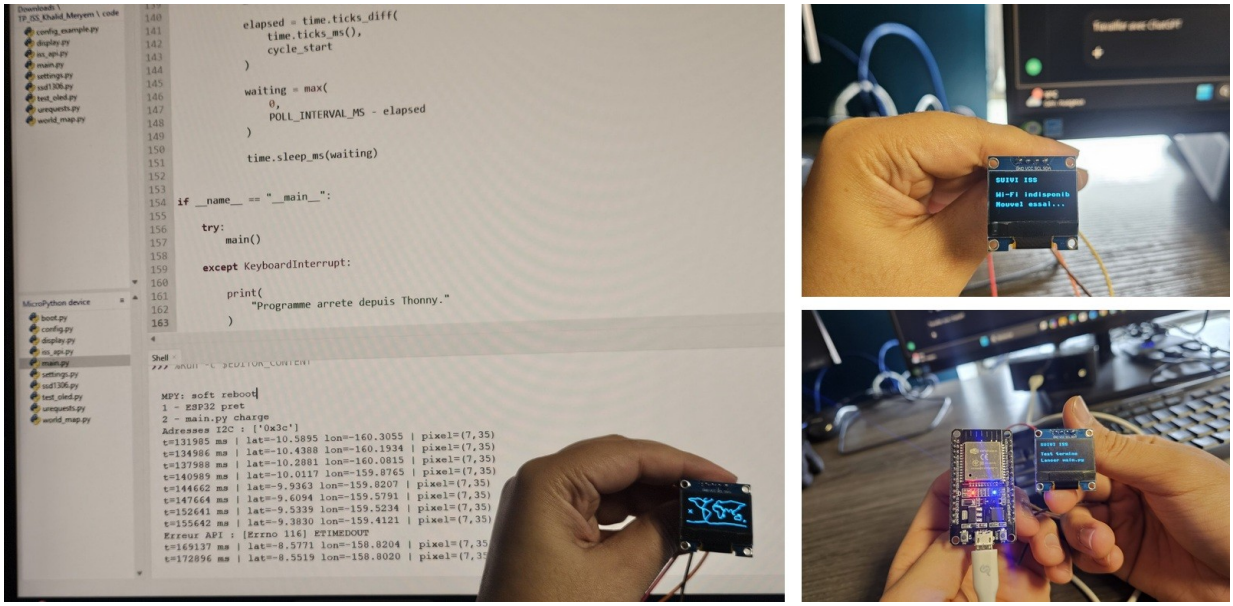


Figure 5 — Preuves de test : suivi réel dans Thonny, gestion d'une indisponibilité Wi-Fi et test OLED.

Essai	Résultat observé	État
Scan I2C	OLED détecté à l'adresse 0x3C.	VALIDÉ
Affichage OLED	Carte du monde visible sur le SSD1306.	VALIDÉ
Connexion Wi-Fi	Connexion obtenue après nouvelle tentative.	VALIDÉ
API Open Notify	Latitude / longitude réelles reçues et converties.	VALIDÉ
Mise à jour du X	Pixels observés : (81,37) puis (82,38), etc.	VALIDÉ
Erreur réseau/API	ETIMEDOUT capturé ; reprise automatique ensuite.	VALIDÉ

Problèmes rencontrés et solutions

Problème rencontré	Solution appliquée
Connexion Wi-Fi parfois lente	Délai de connexion porté à 30 s et conservation d'une connexion déjà active.
Requêtes API parfois trop lentes	Délai HTTP de 10 s, try/except et nouvelle tentative au cycle suivant.
Marqueur difficile à distinguer	Contraste renforcé et X dessiné après la carte.
Déplacement peu visible à court terme	Interprétation correcte de la faible résolution : plusieurs positions peuvent partager le même pixel.

CONCLUSION Le projet répond aux objectifs du TP : le matériel fonctionne, la connexion réseau et l'API sont exploitées, les coordonnées sont validées et projetées correctement, et la position de l'ISS est affichée sur une carte OLED. La gestion des erreurs permet au programme de continuer à fonctionner malgré des problèmes temporaires de réseau.

Références : Énoncé du TP ISS (Institut Grasset) ; Open Notify - ISS Location Now ; documentation MicroPython (network.WLAN, SoftI2C, framebuf) ; bibliothèques ssd1306 et urequests fournies avec le projet.